

Software Integration & Maintenance

Fahim Foysal Abak

Lecture, Dept. of CSE

Dhaka International University

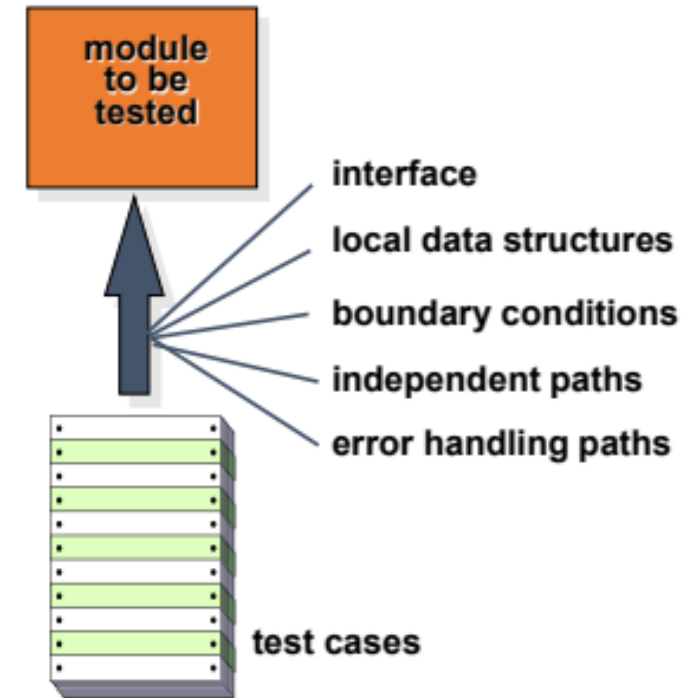
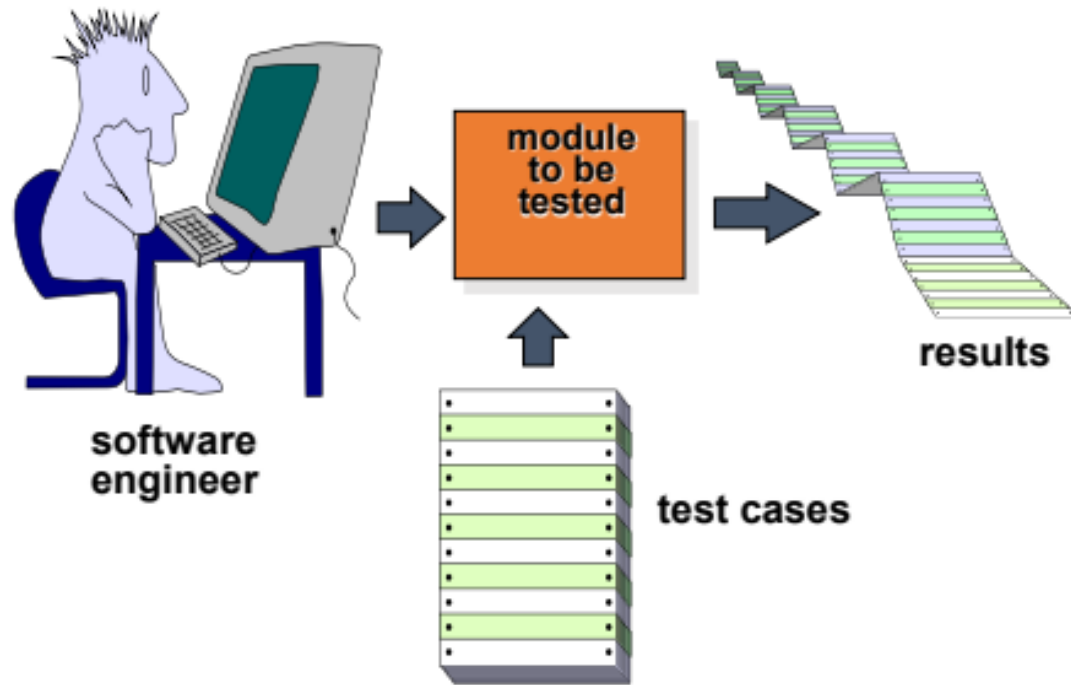
Test Strategies for Conventional Software

- ☐ **Unit Testing**
- ☐ **Integration Testing**
- ☐ **Regression Testing**
- ☐ **Smoke Testing**

Unit Testing

- ❑ Focuses testing on the function or software module
- ❑ Concentrates on the internal processing logic and data structures
- ❑ Is simplified when a module is designed with high cohesion
 - ❑ Reduces the number of test cases
 - ❑ Allows errors to be more easily predicted and uncovered
- ❑ Concentrates on critical modules and those with high cyclomatic complexity when testing resources are limited

Unit Testing



Targets for Unit Test Cases

Module interface

Ensure that information flows properly into and out of the module

Local data structures

Ensure that data stored temporarily maintains its integrity during all steps in an algorithm execution

Boundary conditions

Ensure that the module operates properly at boundary values established to limit or restrict processing

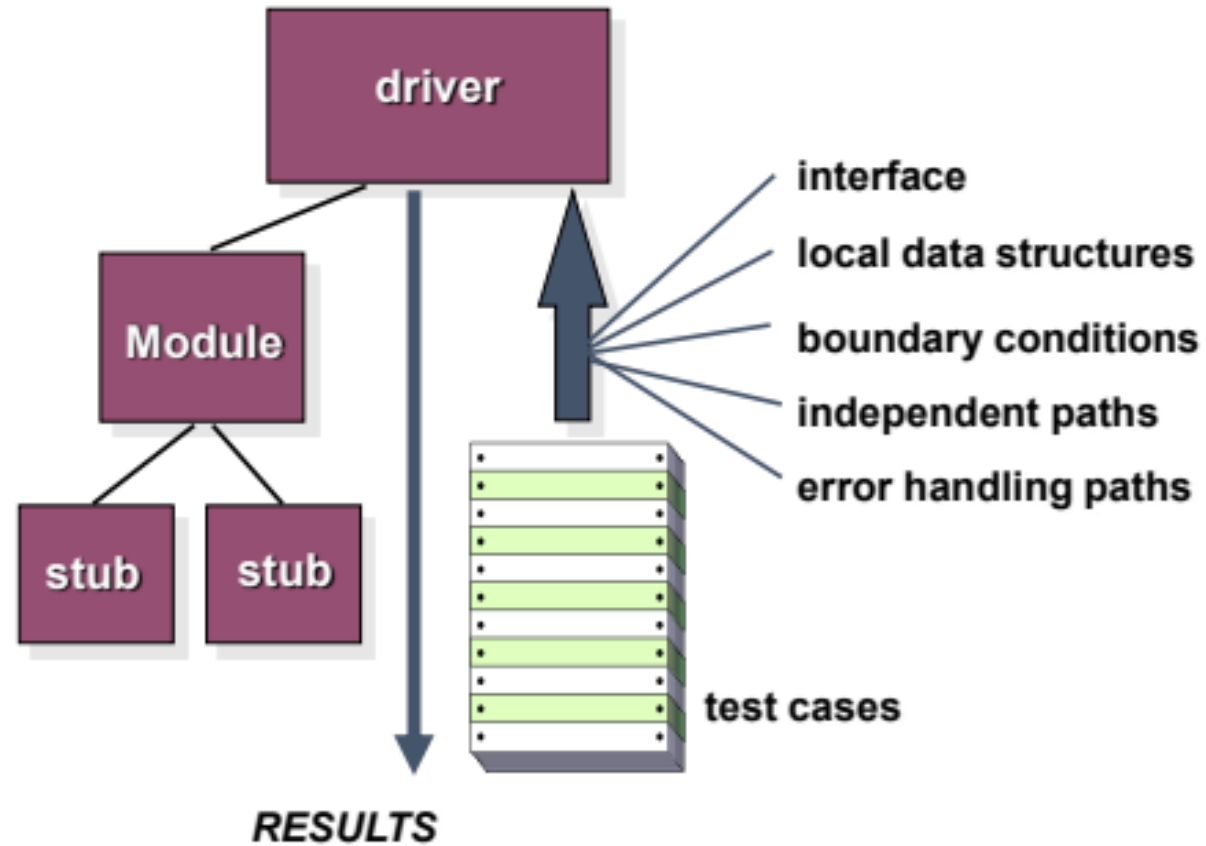
Independent paths (basis paths)

Paths are exercised to ensure that all statements in a module have been executed at least once

Error handling paths

Ensure that the algorithms respond correctly to specific error conditions

Unit Test Environment



Drivers and Stubs for Unit Testing

Driver

A simple main program that accepts test case data, passes such data to the component being tested, and prints the returned results

Stubs

Serve to replace modules that are subordinate to (called by) the component to be tested

It uses the module's exact interface, may do minimal data manipulation, provides verification of entry, and returns control to the module undergoing testing

Drivers and stubs both represent overhead

Both must be written but don't constitute part of the installed software product

Integration Testing

Defined as a systematic technique for constructing the software architecture

- At the same time integration is occurring, conduct tests to uncover errors associated with interfaces

Objective is to take unit tested modules and build a program structure based on the prescribed design

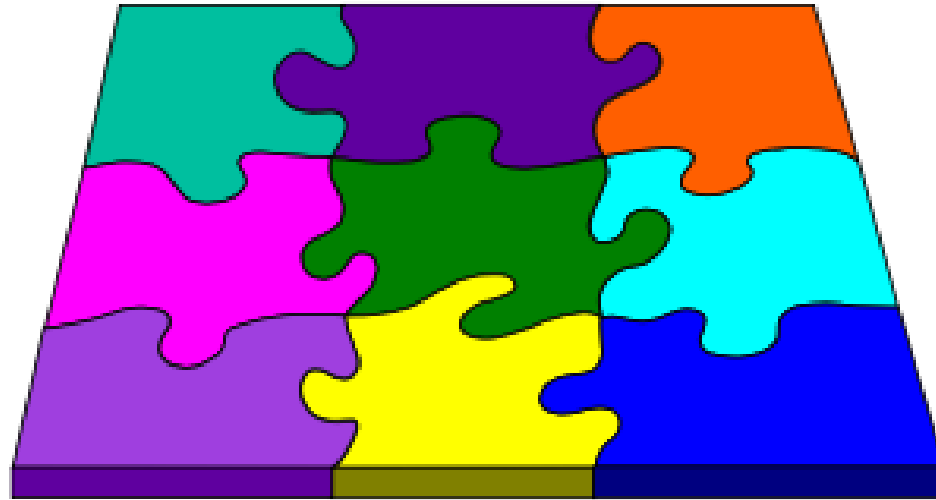
Two Approaches

- Non-incremental Integration Testing
- Incremental Integration Testing

Integration Testing Strategies

Options:

- the “big bang” approach
- an incremental construction strategy



Incremental Integration Testing

Three kinds

- **Top-down integration**
- **Bottom-up integration**
- **Sandwich integration**
- The program is constructed and tested in small increments
- Errors are easier to isolate and correct
- Interfaces are more likely to be tested completely
- A systematic test approach is applied

Top-down Integration

Top-Down Integration

Modules are integrated by moving downward through the control hierarchy, beginning with the main module.

Subordinate modules are incorporated in either a depth-first or breadth-first fashion:

- **DF (Depth-First):** All modules on a major control path are integrated.
- **BF (Breadth-First):** All modules directly subordinate at each level are integrated.

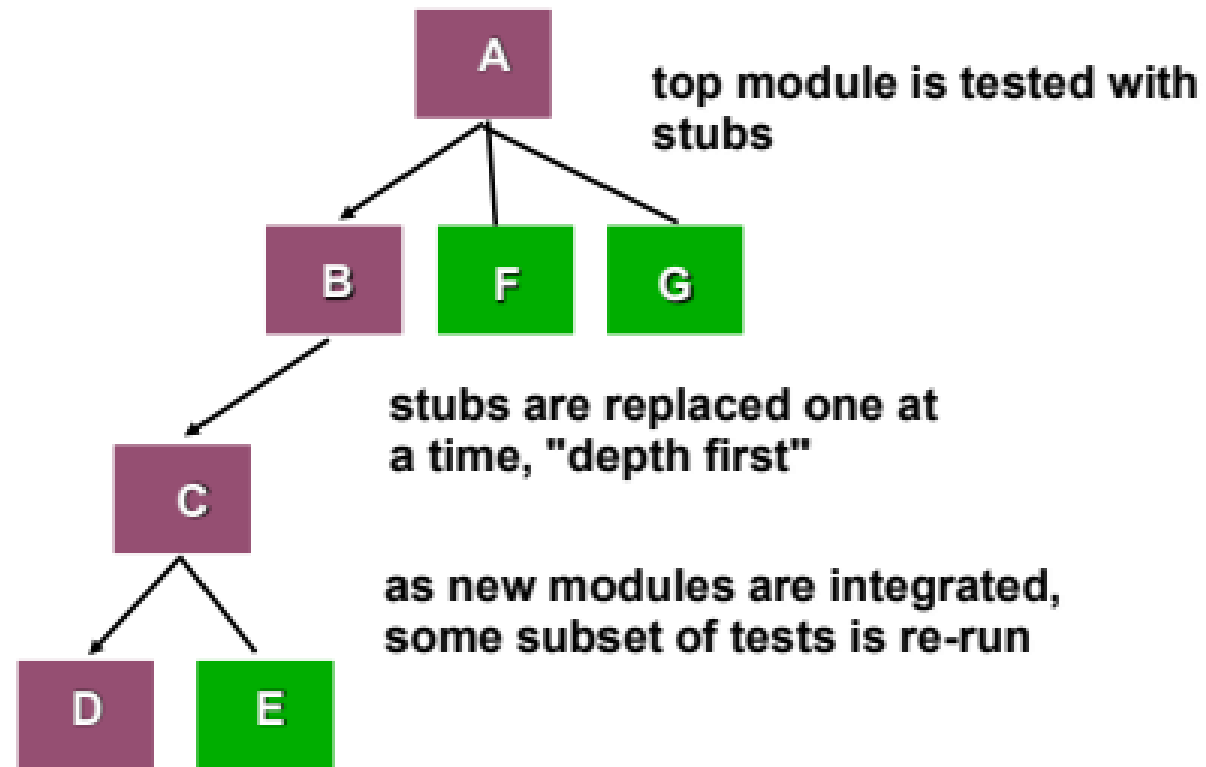
Advantages

- ❑ This approach verifies major control or decision points early in the test process.

Disadvantages

- ❑ Stubs need to be created to substitute for modules that have not been built or tested yet; this code is later discarded.
- ❑ Because stubs are used to replace lower-level modules, no significant data flow can occur until much later in the integration/testing process.

Top-down Integration



Bottom-up Integration

Integration and testing start with the most atomic (lowest-level) modules in the control hierarchy.

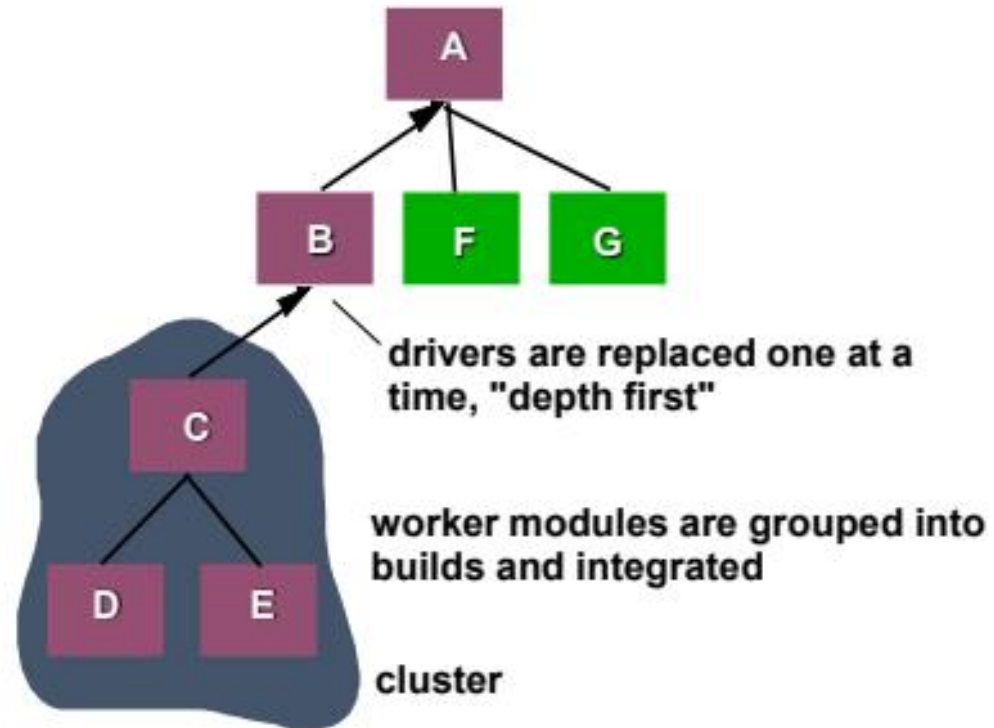
Advantages

- ❑ This approach verifies low-level data processing early in the testing process.
- ❑ The need for stubs is eliminated.

Disadvantages

- ❑ Driver modules need to be built to test the lower-level modules; this code is later discarded or expanded into a full-featured version.
- ❑ Drivers do not contain the complete algorithms that will eventually use the services of the lower-level modules; therefore, testing may be incomplete or additional testing may be required later when upper-level modules become available.

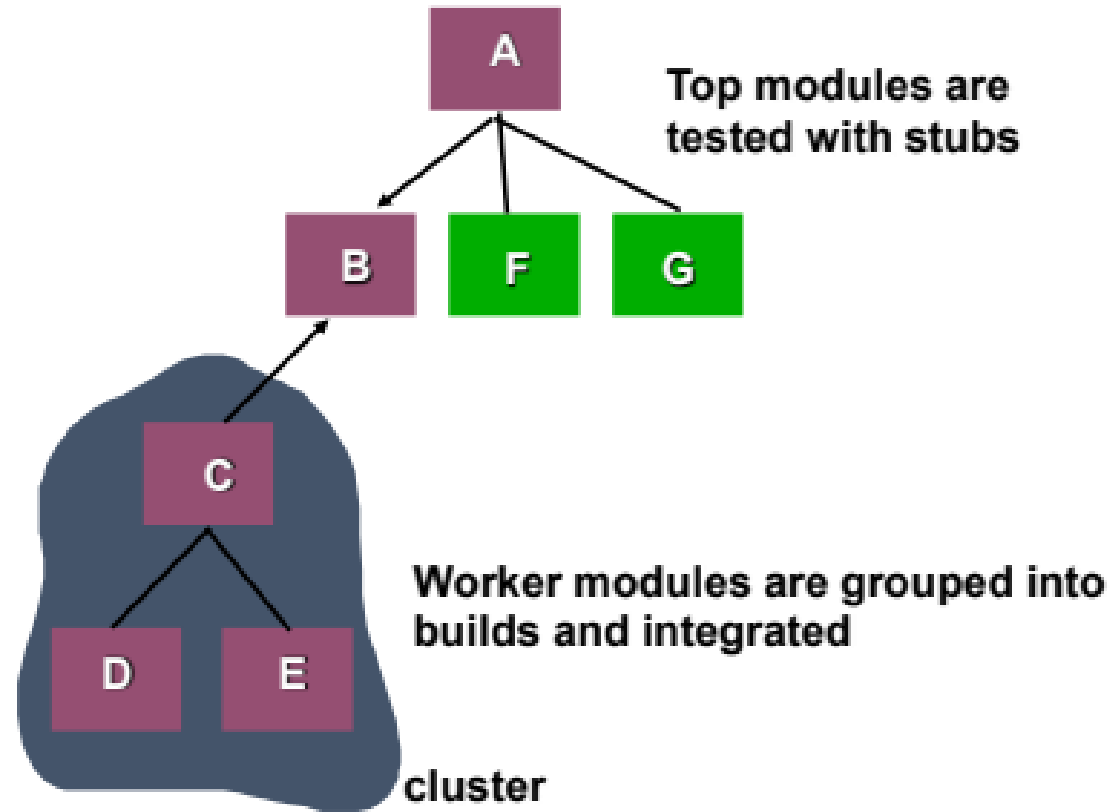
Bottom-up Integration



Sandwich Integration

- ❑ Combines both **top-down** and **bottom-up**
- ❑ integration approaches
- ❑ Applied to **both high-level and low-level modules**
- ❑ Uses **functional groups of modules**, completing one group before moving to the next
- ❑ Groups are based on **control and data processing** for specific program features
- ❑ Integration within a group alternates between **high-level and low-level modules**
- ❑ Once a functional group is complete, testing moves to the **next group**
- ❑ Gains advantages of both top-down and bottom-up while **minimizing drivers and stubs**
- ❑ Requires a **disciplined approach** to avoid a “big bang” scenario

Sandwich Testing



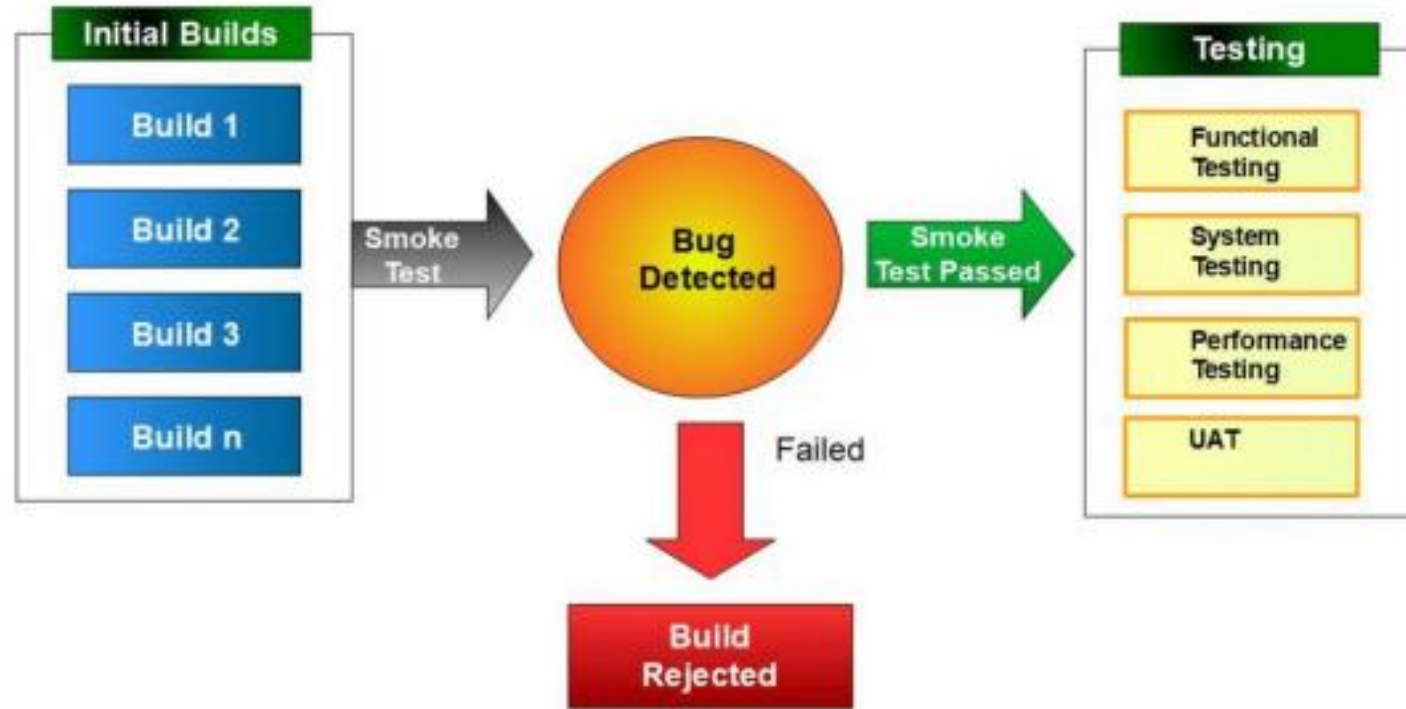
Regression Testing

- ❑ **Definition:** Re-execution of selected tests to ensure changes do not introduce unintended side effects
- ❑ **Purpose:** Ensures that corrections or changes in software, documentation, or supporting data do not create new errors
- ❑ **Execution:**
 - ❑ Can be conducted **manually** by re-running a subset of test cases
 - ❑ Can use **automated capture/playback tools** for efficiency

Smoke Testing

- ❑ **Daily Builds:** A common approach to assemble product software regularly
- ❑ **Smoke Testing Steps:**
 - ❑ Integrate translated software components into a **build**
 - ❑ Build includes all **data files, libraries, reusable modules, and engineered components** needed for product functions
 - ❑ Conduct tests designed to **expose errors** that prevent the build from functioning correctly
 - ❑ Focus on uncovering “**show stopper**” errors that could delay the project
 - ❑ Integrated builds are **smoke tested daily** as part of the full product
 - ❑ Integration can follow **top-down or bottom-up** approach

Smoke Testing



Benefits of Smoke Testing

- ❑ **Minimizes integration risk** by catching issues early
- ❑ **Early detection** of incompatibilities and show-stopper errors reduces schedule impact
- ❑ **Improves overall product quality**
- ❑ **Uncovers errors** at functional, architectural, and component levels
- ❑ **Simplifies error diagnosis and correction**
- ❑ Likely to reveal errors in the **newest integrated components**
- ❑ **Easier progress assessment** as more software is integrated and verified
- ❑ **Managers gain visibility** into project advancement

Test Strategies for Object-Oriented Software

- ❑ With object-oriented software, you can no longer test a single operation in isolation (conventional thinking)
- ❑ Traditional top-down or bottom-up integration testing has little meaning
- ❑ Class testing for object-oriented software is the equivalent of unit testing for conventional software
- ❑ Focuses on operations encapsulated by the class and the state behavior of the class

Test Strategies for Object-Oriented Software

Drivers can be used

- To test operations at the lowest level and for testing whole groups of classes
- To replace the user interface so that tests of system functionality can be conducted prior to implementation of the actual interface

Stubs can be used

- In situations in which collaboration between classes is required but one or more of the collaborating classes has not yet been fully implemented

Test Strategies for Object-Oriented Software

Two different object-oriented testing strategies

Thread-based testing

- Integrates the set of classes required to respond to one input or event for the system
- Each thread is integrated and tested individually
- Regression testing is applied to ensure that no side effects occur

Use-based testing

- First tests the independent classes that use very few, if any, server classes
- Then the next layer of classes, called dependent classes, are integrated
- This sequence of testing layer of dependent classes continues until the entire system is constructed

Background of validation testing

Validation testing follows integration testing

- The distinction between conventional and object-oriented software disappears
- Focuses on user-visible actions and user-recognizable output from the system
- Demonstrates conformity with requirements

Designed to ensure that

- All functional requirements are satisfied
- All behavioral characteristics are achieved
- All performance requirements are attained
- Documentation is correct
- Usability and other requirements are met (e.g., transportability, compatibility, error recovery, maintainability)

Background of validation testing

After each validation test

- The function or performance characteristic conforms to specification and is accepted
- A deviation from specification is uncovered and a deficiency list is created
- A configuration review or audit ensures that all elements of the software configuration have been properly developed, cataloged, and have the necessary detail for entering the support phase of the software life cycle

Alpha and Beta Testing

Alpha testing

- Conducted at the developer's site by end users
- Software is used in a natural setting with developers watching intently
- Testing is conducted in a controlled environment

Beta testing

- Conducted at end-user sites
- Developer is generally not present
- It serves as a live application of the software in an environment that cannot be controlled by the developer
- The end-user records all problems that are encountered and reports these to the developers at regular intervals
- After beta testing is complete, software engineers make software modifications and prepare for release of the software product to the entire customer base

Different Types of system testing

Recovery testing

- Tests for recovery from system faults
- Forces the software to fail in a variety of ways and verifies that recovery is properly performed
- Tests re-initialization, check pointing mechanisms, data recovery, and restart for correctness

Security testing

- Verifies that protection mechanisms built into a system will, in fact, protect it from improper access

Different Types of system testing

Stress testing

- Executes a system in a manner that demands resources in abnormal quantity, frequency, or volume

Performance testing

- Tests the run-time performance of software within the context of an integrated system
- Often coupled with stress testing and usually requires both hardware and software instrumentation
- Can uncover situations that lead to degradation and possible system failure

Characteristics of Testable Software

- **Operable**
 - The better it works (i.e., better quality), the easier it is to test
- **Observable**
 - Incorrect output is easily identified; internal errors are automatically detected
- **Controllable**
 - The states and variables of the software can be controlled directly by the tester
- **Decomposable**
 - The software is built from independent modules that can be tested independently

Characteristics of Testable Software

- **Simple**
 - The program should exhibit functional, structural, and code simplicity
- **Stable**
 - Changes to the software during testing are infrequent and do not invalidate existing tests
- **Understandable**
 - The architectural design is well understood; documentation is available and organized

Test Characteristics

- **A good test has a high probability of finding an error**
 - The tester must understand the software and how it might fail
- **A good test is not redundant**
 - Testing time is limited; one test should not serve the same purpose as another test
- **A good test should be “best of breed”**
 - Tests that have the highest likelihood of uncovering a whole class of errors should be used
- **A good test should be neither too simple nor too complex**
 - Each test should be executed separately; combining a series of tests could cause side effects and mask certain errors

Test Case Design

Bugs lurk in corners and congregate at boundaries.

- Boris Beizer

OBJECTIVE

to uncover errors

CRITERIA

in a complete manner

CONSTRAINT

with a minimum of effort and time

Thank You.